The Kernel

2026 Semester 2 COMPSCI 340: Operating Systems
Talia Xu

Lecture 2
2.2.1

Based on original slides by Professor Jon Eyolfson.

Three Ideas Organize the Course

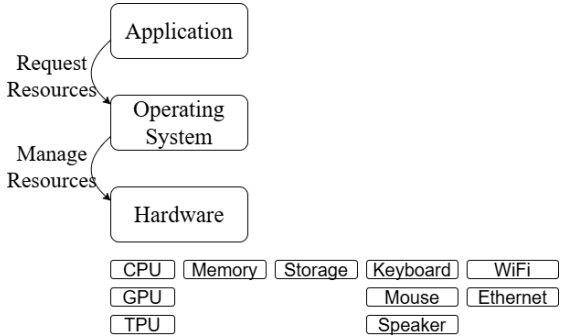
Virtualization
gives each program a simpler view of shared hardware

Concurrency
lets multiple activities make progress over the same resources

Persistence
keeps data available after a program or machine stops

This lecture continues with virtualization: the OS gives programs simple views while managing the shared machine underneath.

Shared Hardware Becomes Useful OS Abstractions



- CPU time becomes processes and threads
- Memory becomes private address spaces
- Devices and storage become named resources

A Process Is a Running Program Plus State

Program

- Instructions and static data
- Stored in a file
- Not currently executing

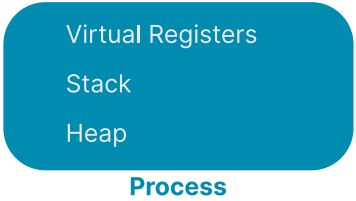
Process

- A program being executed
- Registers and memory
- OS-managed resources and permissions

A Process Needs CPU State and Memory State

To pause and resume a process, the OS must preserve:

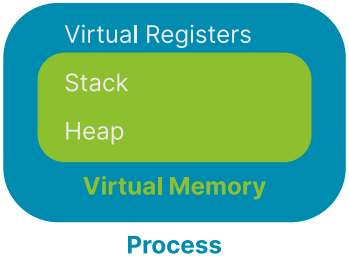
- Registers and program counter
- Stack, heap, code, and data
- Open resources and other metadata



Running the Same Program Twice

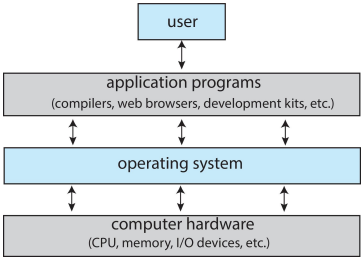
```c
#include <stdio.h>
#include <unistd.h>

static int global = 0;

int main(void) {
  int local = 0;
  while (1) {
    ++local;
    ++global;
    printf("local = %d, global = %d\n", local, global);
    printf("0x%lx, 0x%lx\n", (long int)&local, (long int)&global);
    sleep(1);
  }
  return 0;
}
```

Virtual Memory Gives Each Process Its Own Address Space



- Each process uses its own virtual addresses
- Hardware and the kernel map them to physical memory
- Matching virtual addresses need not share physical storage

OS Abstractions Need a Privileged Kernel



Applications cannot enforce isolation or resource sharing themselves.

The **kernel** can:

- Manage shared hardware and state
- Enforce process isolation
- Decide whether requests are permitted

Why Does the CPU Need Two Modes?

User mode

- Runs applications
- Restricts privileged instructions
- Restricts access to kernel memory

Kernel mode

- Runs trusted OS code
- Can control devices and memory
- Can affect the whole system

The CPU enforces this boundary; an application cannot simply grant itself privilege.

Most Application Instructions Stay in User Mode

```
total += values[i];
```

Arithmetic, branches, and access to a process's own memory execute directly in user mode.

The CPU enters kernel code only when an event requires OS involvement:

- A system call
- An exception
- A hardware interrupt

What If Applications Had Kernel Privileges?

If every application ran with kernel privileges, what could happen if:

- A browser wrote to any memory address it chose?
- A game disabled the timer that lets the OS regain the CPU?
- Two programs issued conflicting commands to a disk?

Three Events Transfer Control to the Kernel

System call
an application deliberately requests an OS service

Exception
the current instruction needs attention, such as a page fault or divide-by-zero

Interrupt
hardware signals an event, such as a timer tick or completed device operation

The kernel handles the event, then either resumes the process or takes another action.

Event 1: A System Call Is a Deliberate Request

```c
#include <unistd.h>

int main(void) {
    write(STDOUT_FILENO, "hello\n", 6);
    return 0;
}
```

Event 1: A System Call Is a Deliberate Request

```c
#include <unistd.h>

int main(void) {
    write(STDOUT_FILENO, "hello\n", 6);
    return 0;
}
```

`write()` takes a file descriptor, the address of some bytes, and a byte count.

Calling it deliberately requests an OS service. The kernel performs the output and returns a result.

Event 2: An Instruction Causes an Exception

```c
int main(void) {
    int *value = 0;
    *value = 42;
    return 0;
}
```

Event 2: An Instruction Causes an Exception

```c
int main(void) {
    int *value = 0;
    *value = 42;
    return 0;
}
```

Dereferencing `value` has undefined behaviour in C.

On a typical protected system, the CPU raises a page-fault exception. The kernel handles it, usually terminating the process with `Segmentation fault`.

Event 3: An Interrupt Arrives from Hardware

```
int main(void) {
    for (;;) {
        /* keep computing */
    }
}
```

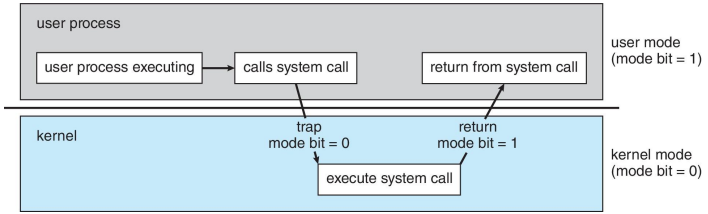
Event 3: An Interrupt Arrives from Hardware

```c
int main(void) {
    for (;;) {
        /* keep computing */
    }
}
```

In C, `for (;;)` means loop forever. The loop makes no system call and causes no exception.

Timer hardware still interrupts the CPU periodically. The kernel may resume this process or schedule another one.

Applications Request Kernel Services with System Calls



A **system call** transfers control to an approved kernel entry point. The kernel validates the request, performs the operation, and returns a result.

System calls cover operations such as files and devices, process creation, memory mapping, and communication.

A Trap Transfers Control to the Kernel

A **trap** is how the CPU pauses the current code and starts the correct kernel code.

During setup or boot

Kernel	Tells the CPU where the handler starts and prepares a kernel stack
CPU	Keeps this information for when a trap occurs

When a trap occurs

Trigger	A system call, exception, or interrupt occurs
CPU	Pauses the current code, remembers where it stopped, enters kernel mode, and starts the handler
Kernel	Saves the process state, handles the event, and decides what should run next
Kernel and CPU	Restore the saved state and continue the program in user mode

A System Call Uses Software and Hardware Conventions

```
ssize_t result = write(1, "Hi\n", 3);
```

API
names the operation and defines its argument and result types

ABI
defines the system-call number and locations of arguments and results

ISA
provides the controlled entry instruction: `syscall`, `svc`, or `ecall`

File Descriptors Name Per-Process Resources

A **file descriptor** is a small integer indexing a resource held by a process.

By convention:

- `0` is standard input
- `1` is standard output
- `2` is standard error

A descriptor may refer to a terminal, file, pipe, device, or socket; calls such as `read()`, `write()`, and `close()` use the same abstraction.

Minimal Hello World Needs Output and Exit

```
ssize_t write(int fd, const void *buf, size_t count);
void exit_group(int status);
```

Even without the C standard library, a program needs OS services to produce output and terminate.

`strace` Shows Requests to the Kernel

```
strace ./hello_world
```

Each trace line has the form: **request(arguments)** = **result**.

A C Program Makes Different Kernel Requests

```
execve("./hello_world_c", ...)          = 0
openat(..., "/usr/lib/libc.so.6", ...) = 3
mmap(..., fd=3, ...)                    = ...
brk(...)                                = ...
write(1, "Hello world\n", 12)           = 12
exit_group(0)                           = ?
```

A normal C program also needs work from the loader and C runtime.

Operating Systems Make Different Design Choices

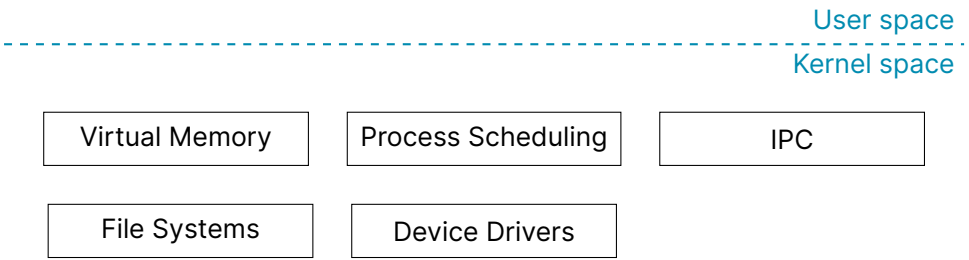
Windows, Linux, macOS, Android, iOS, and ChromeOS all manage similar hardware.

They differ in choices such as:

- Which abstractions and interfaces they expose
- Which policies decide how resources are shared
- How much operating-system code runs with full privilege

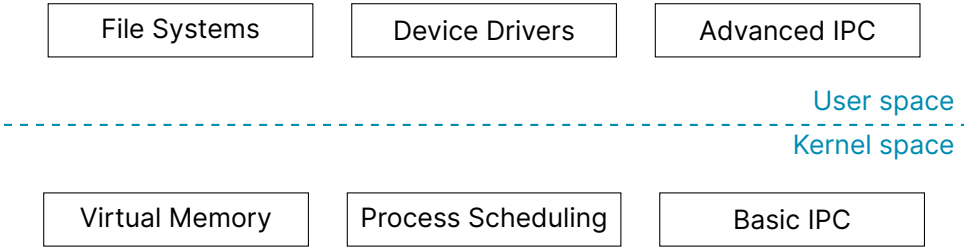
The placement of privileged services is a kernel architecture decision.

Monolithic: More Services Share Kernel Space



Direct function calls can be efficient, but more code shares privilege and a failure domain.

Microkernel: Keep the Privileged Core Small



Isolated user-space services reduce the privileged code base, but communicate through messages.

Kernel Structure Is a Design Trade-off

More in kernel space

- Direct communication
- Lower boundary-crossing cost
- Larger trusted code base

More in user space

- Better service isolation
- More message passing
- Smaller privileged core

Real systems often use hybrid choices. The useful question is **which services need privilege, and what cost does that choice create**.

`printf()` and `write()`

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("1");
    sleep(1);
    write(1, "2", 1);
    return 0;
}
```

Summary

- The kernel manages shared processors, memory, devices, and system state
- CPU modes let trusted kernel code enforce isolation from applications
- System calls let applications request OS services in a controlled way
- Kernel architectures choose which services run with privilege